

AI-FITTRACK

PHASE 3- PROJECT DESIGN

1. DESIGN OVERVIEW

Project design translates the verified requirements of AI FitTrack into a structured technical design. The design covers the backend architecture, API route organization, database entities, user flow, MVC responsibilities, data-flow representation, module boundaries, and security-related design considerations.

The design reflects the supplied implementation: a Node.js and Express.js backend using MongoDB/Mongoose, JWT authentication, bcryptjs password protection, and a dedicated Google Gemini service. The database design contains User and Workout models.

2. TECHNICAL ARCHITECTURE

The system follows a layered backend architecture. An API consumer sends HTTP requests to the Express server. Requests are mapped to route modules, protected routes pass through JWT authentication middleware, and controllers perform business processing. Controllers interact with Mongoose models for database operations or the Gemini service for AI operations.

Express application-level responsibilities also include CORS support, JSON request parsing, HTTP request logging, a health endpoint, route registration, a not-found handler, and centralized error handling.

3. ARCHITECTURE COMPONENTS

Component	Design Responsibility
Client / API Consumer	Sends HTTP requests and receives JSON responses. Thunder Client/Postman can be used as API consumers for verification.
Express Server	Runs the backend application and receives HTTP requests.
API Routes	Groups authentication, workout, and AI endpoints and maps requests to controllers.
JWT Authentication Middleware	Verifies JWT tokens for protected operations and attaches authenticated user information to the request.
Controllers	Contain application/business processing for authentication, workouts, and AI operations.
Mongoose Models	Represent the User and Workout data structures and provide database interaction.
MongoDB Atlas	Stores User and Workout records.
Gemini Service	Sends recommendation and insight requests to Google Gemini.
Error Middleware	Provides centralized handling for application errors and unmatched routes.

4. TECHNICAL ARCHITECTURE DIAGRAM

Figure 1 presents the architecture of the verified implementation. The AI path is separate from the MongoDB data path: workout data is handled through Mongoose, while AI requests are delegated to the Gemini service.

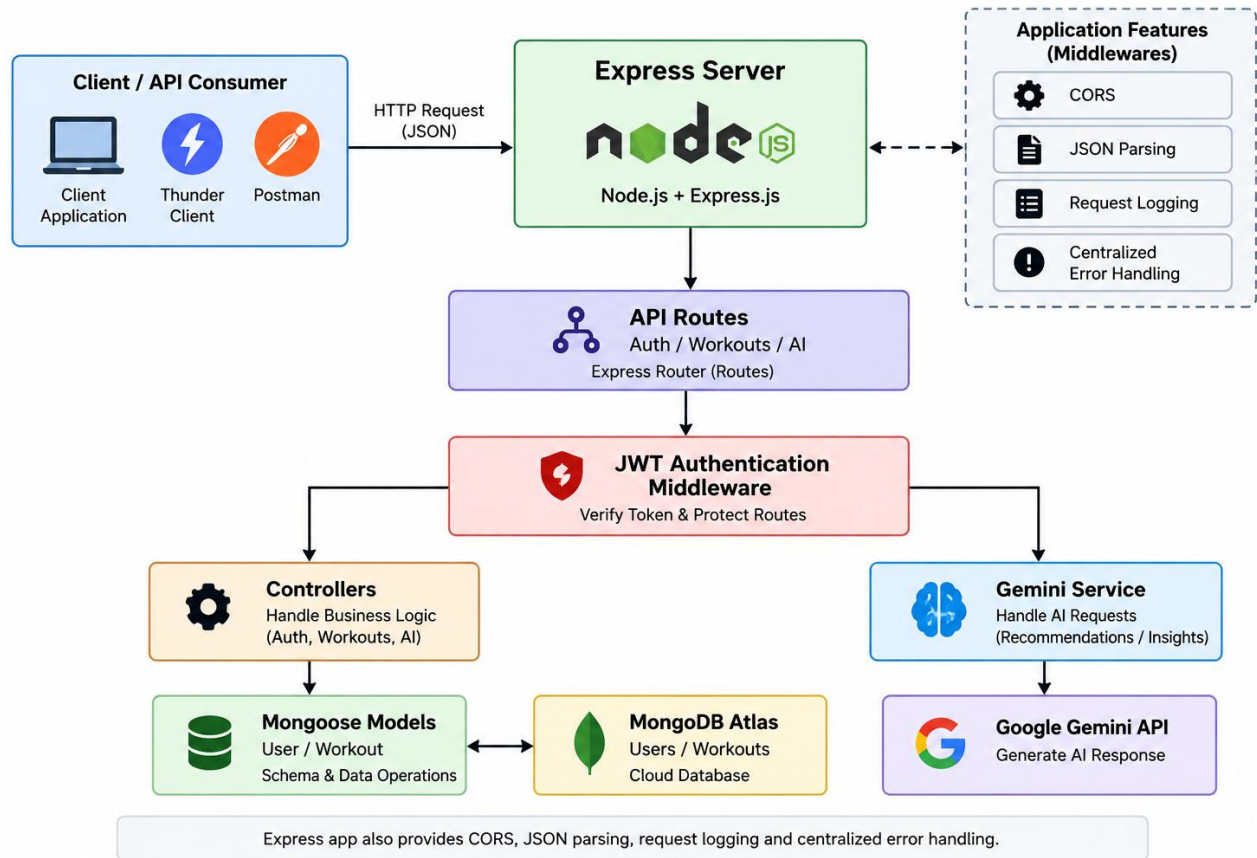


Figure 1: Technical Architecture of AI FitTrack

5. API ROUTE DESIGN

Module	Method	Endpoint	Access / Purpose
Authentication	POST	/api/auth/register	Public – register a user
Authentication	POST	/api/auth/login	Public – authenticate and receive JWT
Authentication	GET	/api/auth/profile	Protected – retrieve authenticated profile
Workout	POST	/api/workouts	Protected – create workout
Workout	GET	/api/workouts	Protected – retrieve user's workouts
Workout	GET	/api/workouts/search	Protected – search by name, category, or date
Workout	GET	/api/workouts/:id	Protected – retrieve a workout by ID

Module	Method	Endpoint	Access / Purpose
Workout	PUT	/api/workouts/:id	Protected – update workout
Workout	DELETE	/api/workouts/:id	Protected – delete workout
AI	POST	/api/ai/workout-recommendation	Protected – request workout recommendation
AI	POST	/api/ai/fitness-insights	Protected – request fitness insights

6. ER DIAGRAM AND DATABASE DESIGN

The verified database design contains two primary entities: User and Workout. A User can have multiple Workout records. The Workout document stores a reference to its owning User through an ObjectId.

Entity	Fields
User	_id, name, email, password, createdAt, updatedAt
Workout	_id, user, workoutName, category, duration, caloriesBurned, workoutDate, createdAt, updatedAt

The User email is unique and stored in normalized lowercase form. Passwords are hashed before storage. The Workout category is constrained to the categories implemented by the supplied model, and duration and caloriesBurned have model-level numeric validation.

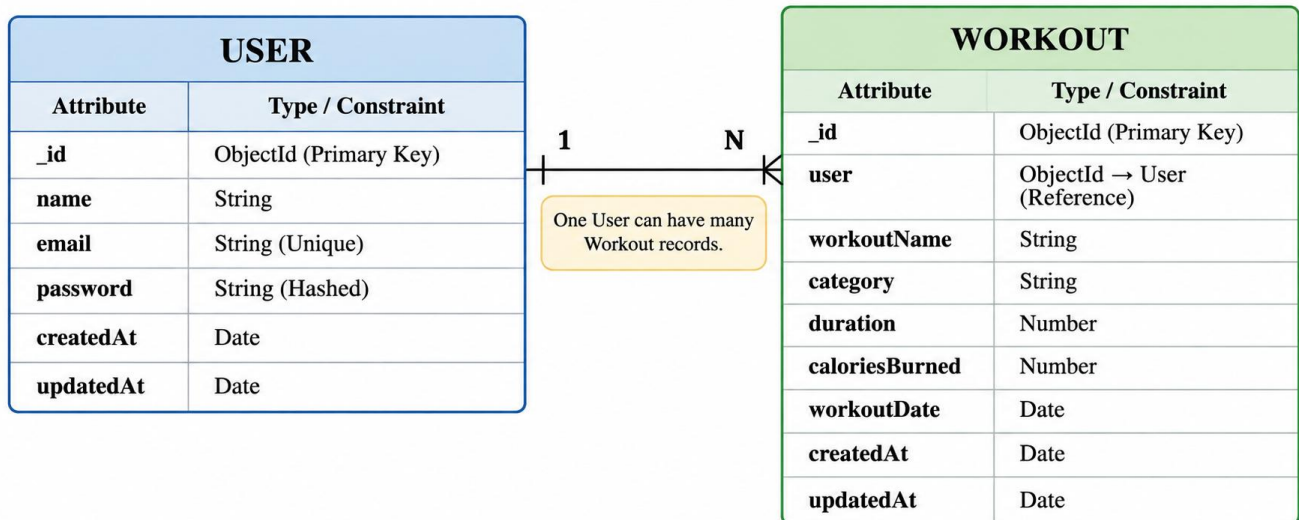


Figure 2: Entity Relationship Diagram

7. USER FLOW DESIGN

The conceptual user flow begins with registration or login. After authentication, the user receives a JWT and can access protected operations. The authenticated user can manage workout records, search existing records, or submit the required inputs to the AI endpoints. The API returns the resulting JSON response.

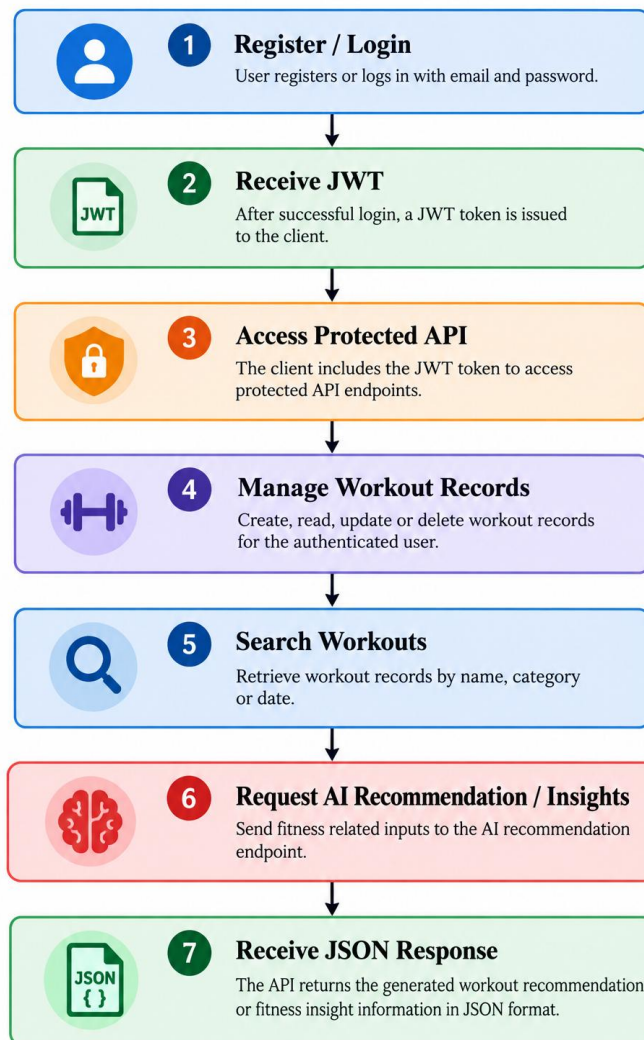


Figure 3: User Flow

8. MVC PATTERN

MVC Element	AI FitTrack Responsibility
Model	Mongoose models represent User and Workout data and interact with MongoDB.
View	The API consumer acts as the view/client layer by sending requests and inspecting JSON responses.
Controller	Authentication, workout, and AI controllers implement application processing and coordinate models or the Gemini service.

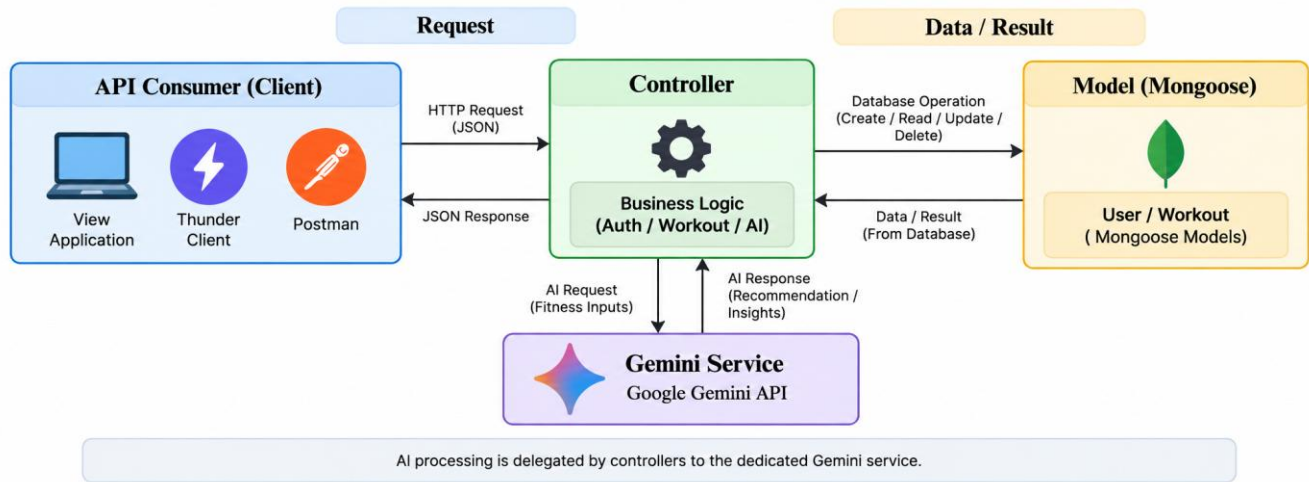


Figure 4: MVC Pattern

9. DATA FLOW DIAGRAM – LEVEL 0

At Level 0, the API consumer interacts with the AI FitTrack REST API. The backend exchanges User and Workout data with MongoDB Atlas and sends AI requests to Google Gemini, returning generated results to the API consumer.

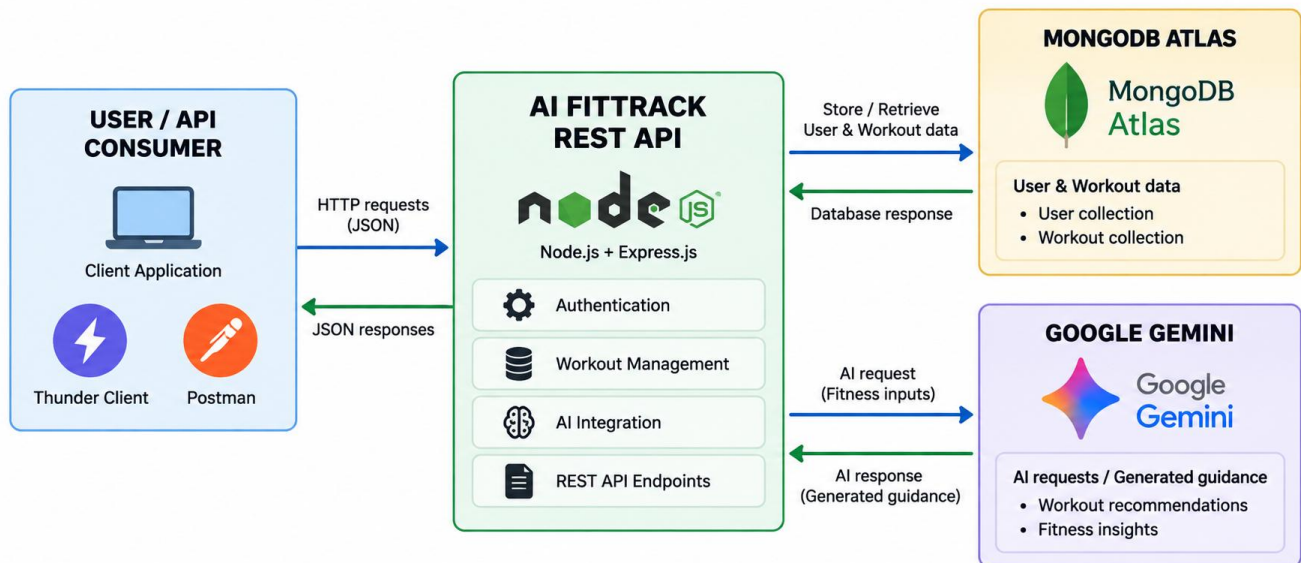


Figure 5: DFD Level 0

10. DATA FLOW DIAGRAM – LEVEL 1

Level 1 decomposes the system into authentication, workout management, workout search, and AI assistance processes. User data is maintained in the Users data store, workout records are maintained in the Workouts data store, and AI requests are passed to the external Google Gemini service.

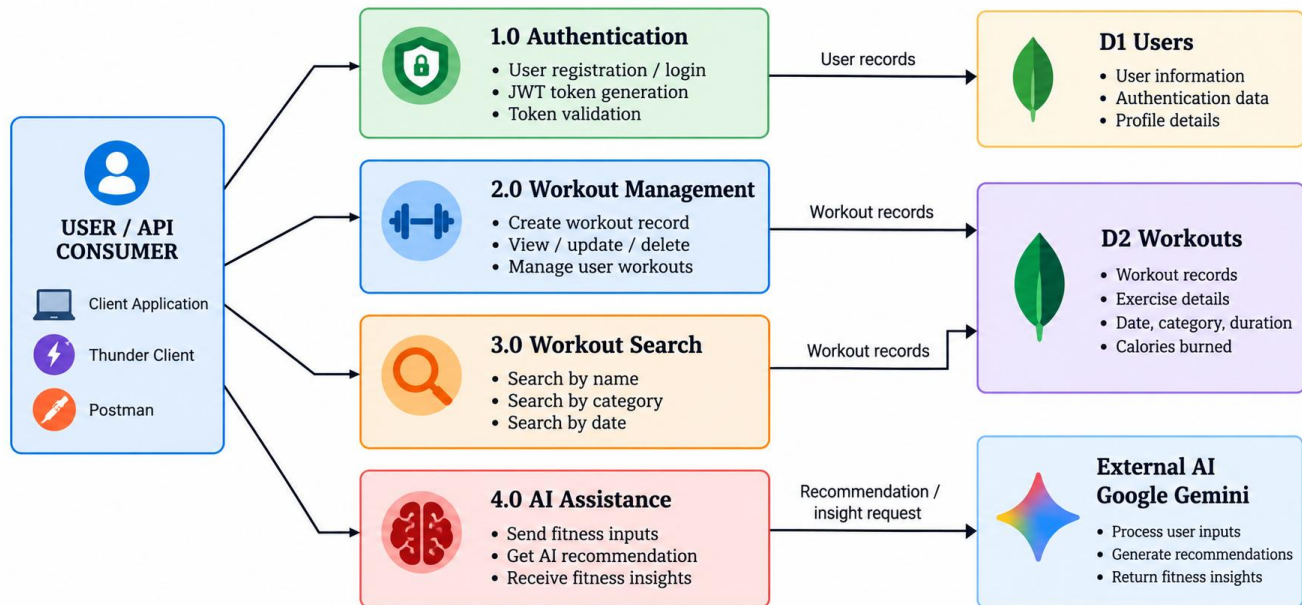


Figure 6: DFD Level 1

11. MODULE DESIGN

Module	Main Files	Design Responsibility
Application	src/app.js, src/server.js	Express application configuration, middleware, route mounting, server startup.
Configuration	src/config/db.js	MongoDB connection configuration.
Authentication	authRoutes.js, authController.js, auth.js, User.js	Registration, login, profile access, JWT verification, password hashing/comparison.
Workout	workoutRoutes.js, workoutController.js, Workout.js	Workout creation, retrieval, search, update, and deletion.
AI	aiRoutes.js, aiController.js, geminiService.js	Recommendation and fitness-insight requests through Google Gemini.
Error Handling	errorHandler.js	Centralized application error handling.

12. SECURITY DESIGN

Security Area	Design
Authentication	JWT tokens are used to protect secured API routes.
Password Protection	bcryptjs is used to hash passwords and compare login passwords.
Ownership Control	Workout retrieval, update, and deletion are performed in the context of the authenticated user.
Secret Management	MongoDB URI, JWT secret, and Gemini API key are supplied through environment variables rather than source code.
API Separation	Authentication, workout, and AI functions are separated into route and controller modules.
Repository Protection	The .env file is excluded through .gitignore and should not be committed to GitHub.

13. DESIGN SUMMARY

The Project Design phase establishes the technical structure required to implement and demonstrate the verified AI FitTrack requirements. The architecture, database relationship, API routes, MVC responsibilities, data flow, module boundaries, and security design all correspond to the supplied backend implementation.